Advanced Spring Boot Task: Secure File Upload Management System

Build a production-grade Secure File Upload Management System using Spring Boot. The system should support secure uploads, validations, encrypted storage, audit logging, malware scan integration hooks, and role-based file access. The application must be scalable and capable of handling enterprise-level uploads securely.

Business Scenario:

A fintech platform allows users to upload KYC documents, invoices, contracts, and profile images. The organization requires a secure upload system to prevent malicious files, unauthorized access, and sensitive data exposure.

Objectives:

1. Implement secure upload/download APIs
2. Validate file size, type, and content
3. Enable encrypted storage support
4. Implement audit tracking
5. Support JWT-based authorization
6. Provide malware scan integration support

Functional Requirements:

1. Upload, download, and delete file APIs
2. Validate MIME type and file extensions
3. Restrict maximum upload size
4. Prevent duplicate uploads
5. Generate unique file identifiers
6. Store metadata in database
7. Encrypt sensitive files
8. Add audit logs for all operations
9. Implement temporary download URLs
10. Use JWT authentication for secure access
11. Support async upload processing
12. Implement upload rate limiting

Technical Requirements:

1. Use Spring Boot layered architecture

2. Use Spring Security with JWT
3. Use MySQL/PostgreSQL for metadata
4. Use Redis for caching
5. Use AWS S3 or local secure storage abstraction
6. Implement exception handling
7. Write unit and integration tests
8. Use asynchronous processing for large files

Deliverables:

1. Working secure file upload application
2. Secure upload/download APIs
3. JWT authentication module
4. Encrypted storage implementation
5. Audit logging implementation
6. Database schema and setup scripts
7. Swagger/Postman documentation
8. Architecture documentation
9. Deployment instructions
10. Testing reports

Expected Architecture Components:

1. API Layer
2. Authentication & Authorization Layer
3. Validation Module
4. Storage Layer
5. Encryption Layer
6. Audit Logging System
7. Asynchronous Processing Layer
8. Monitoring & Metrics

Duration (4–6 Days):

1. Day 1: Architecture design and setup
2. Day 2: Upload/download API implementation
3. Day 3: Security and encryption implementation
4. Day 4: Validation and audit logging
5. Day 5: Async processing and optimization

6. Day 6: Testing, documentation, and review

Evaluation Criteria:

1. Security implementation quality
2. Validation robustness
3. Performance under large file uploads
4. Code quality and modularity
5. Scalability and maintainability
6. Documentation clarity